1. Executive Summary

The Book Exchange Platform is a full-stack web application built for the Software Engineering and Project Management Lab. It supports public browsing of available books, buyer registration and authentication, seller approval by an administrator, seller-managed book listings, buyer purchase or exchange requests, and role-based administration.

The implementation uses Spring Boot 3.5.13, Spring MVC, Thymeleaf, Spring Security, Spring Data JPA, PostgreSQL, Docker, GitHub Actions, and Render. The delivered system includes both a web interface and REST APIs, layered business logic, automated tests, complete Dockerization, and a public cloud deployment.

The final deployed application is available at: https://sepm-book-exchange-platform.onrender.com

2. Objectives and Scope

The project was designed to satisfy the mandatory full-stack and DevOps requirements of the course while keeping the scope realistic for a two-person semester lab project.

2.1 Core Objectives

- Provide separate responsibilities for public visitors, buyers, sellers, and administrators.
- Enforce authentication and role-based authorization with Spring Security.
- Support seller application approval before sellers can create listings.
- Allow sellers to create and manage book listings for selling, exchanging, or both.
- Allow buyers to discover books, search, filter, and submit requests.
- Provide REST APIs alongside the Thymeleaf web application.
- Use PostgreSQL as the persistent data store.
- Support automated testing, Dockerized execution, CI, and Render deployment.

2.2 Delivered Scope

Area	Delivered Capability
Authentication	Buyer registration, custom login/logout, BCrypt password encoding
Role Control	Public visitor, buyer, seller, and admin access separation
Seller Workflow	Seller application submission and admin approval/rejection
Book Management	Seller-side CRUD for listings with category assignment and availability control
Buyer Workflow	Public browse, search, filter, pagination, details page, and request submission
Administration	Category management and seller application

	review
REST API	Books, categories, and exchange request endpoints with JSON responses
Testing	Service unit tests, repository tests, and MockMvc integration tests
DevOps	Dockerfile, docker-compose, staged CI, Render deployment

3. System Overview

3.1 User Roles

Role	Access Level	Key Actions
Public Visitor	Unauthenticated	Browse available books, open details pages, view milestones
Buyer	Authenticated	Register/login, submit seller application, submit book requests, track own requests
Seller	Authenticated + approved seller role	Create/edit/delete listings, manage incoming requests
Admin	Authenticated + admin role	Review seller applications, manage categories, access admin dashboard

3.2 Major Functional Modules

- **Authentication and Authorization:** Custom login flow, registration flow, BCrypt password storage, URL-based access control.
- **Seller Approval Module:** A buyer can apply to become a seller, and an admin can approve or reject the application.
- **Book Listing Module:** Approved sellers create and manage book listings with category, mode, condition, price, and availability.
- **Public Catalog Module:** Visitors and buyers can browse available books, search by title or author, filter by category, and paginate results.
- **Request Workflow Module:** Buyers create BUY or EXCHANGE requests, and sellers approve or reject them.
- **Admin Module:** Admin users manage categories and pending seller applications.
- **REST API Module:** Public and secured JSON endpoints provide book, category, and request operations.
- **Milestone Timeline Module:** A milestone page visualizes merged feature-branch history from local git metadata.

4. System Architecture

The project follows a layered Spring Boot architecture. Thymeleaf-based web controllers and REST controllers sit above the service layer, which contains business rules and

validation. Spring Data JPA repositories handle persistence, while PostgreSQL stores runtime data. Spring Security enforces access control, and Docker plus Render provide deployment support.

4.1 Architecture Diagram

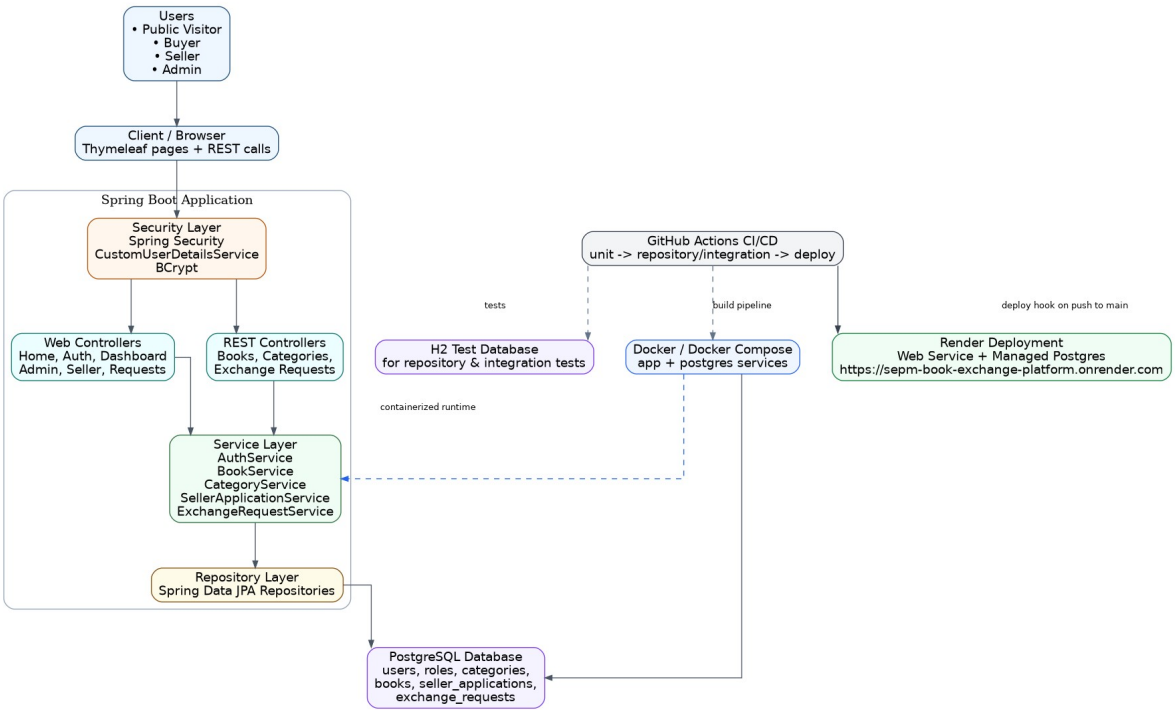

Figure 1. Layered architecture of the Book Exchange Platform

4.2 Architectural Notes

- Thymeleaf templates provide the server-rendered UI for home, authentication, browsing, admin, seller, and request pages.
- REST controllers expose JSON endpoints for books, categories, and exchange requests.
- The service layer centralizes business rules such as duplicate prevention, seller approval, price validation, and request approval effects.
- Repositories are implemented with Spring Data JPA and custom finder methods.
- The runtime database is PostgreSQL, while repository and integration tests use H2 configured in PostgreSQL compatibility mode.
- Docker Compose supports local app + database execution, and GitHub Actions triggers deployment to Render after passing staged tests on main.

5. Database Design

The data model is centered on users, roles, categories, books, seller applications, and exchange requests. Relationships are intentionally simple and explicit, which helps keep the business logic understandable and easier to test.

5.1 ER Diagram

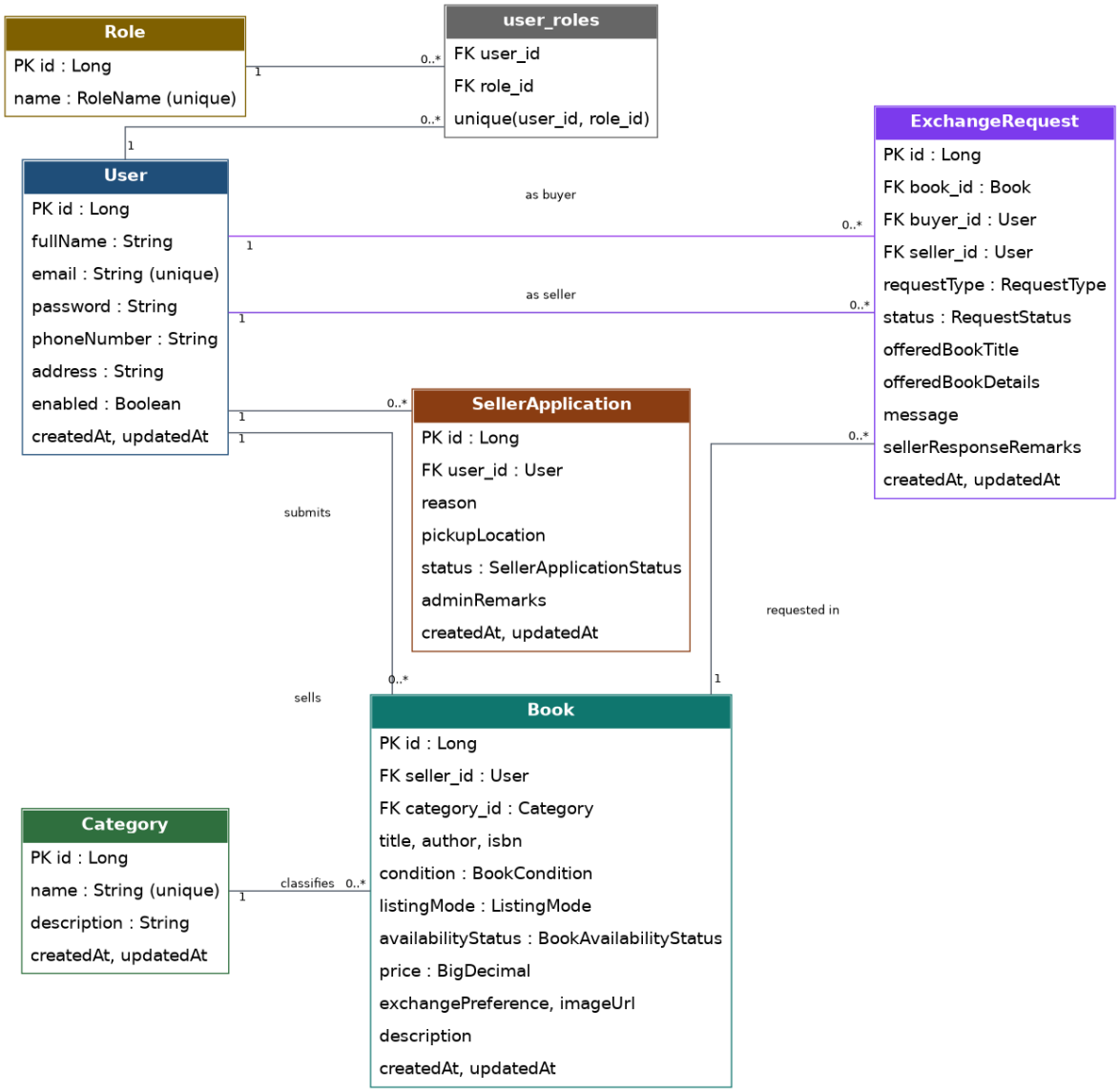

Figure 2. Entity–relationship diagram for the Book Exchange Platform

5.2 Relationship Summary

- User ↔ Role: many-to-many via the user_roles join table.
- Book → User (seller): many-to-one.
- Book → Category: many-to-one.
- SellerApplication → User: many-to-one.
- ExchangeRequest → Book: many-to-one.
- ExchangeRequest → User (buyer): many-to-one.
- ExchangeRequest → User (seller): many-to-one.

6. Implementation Details

6.1 Security and Access Control

- Spring Security uses a custom login page and form-based authentication.
- CustomUserDetailsService loads users by normalized email address.
- BCryptPasswordEncoder protects stored passwords.
- Public endpoints include home, browse pages, milestones, and selected GET REST APIs.
- Seller-only paths protect listing management and incoming request operations.
- Admin-only paths protect category management and seller approval operations.
- CSRF remains enabled for the web app and is ignored only for /api/** to simplify API testing.

6.2 Seller Application Workflow

- A newly registered account starts as a buyer.
- A buyer can submit a seller application with a reason and pickup location.
- Duplicate pending applications are blocked.
- When approved by an admin, ROLE_SELLER is added to the same user account without removing ROLE_BUYER.

6.3 Book Listing Workflow

- Approved sellers create listings with title, author, condition, listing mode, availability, and category.
- Listing modes support exchange-only, sell-only, or both.
- Price is required for SELL_ONLY and BOTH listings.
- Only the seller who owns a listing can update or delete it.

6.4 Buyer Request Workflow

- A buyer can submit BUY or EXCHANGE requests from the public book details page.
- A user cannot request their own listing.
- Duplicate pending requests for the same buyer and book are blocked.
- If a request is approved, the related book becomes unavailable publicly.

7. REST API Overview

The system exposes REST endpoints in parallel with the Thymeleaf web application. Responses use DTOs instead of exposing JPA entities directly, and a global REST exception handler standardizes error responses.

Controller	Endpoint Group	Access	Purpose
BookRestController	GET /api/books, GET /api/books/{id}	Public	Browse public available books as JSON

BookRestController	GET /api/seller/books, POST/PUT/DELETE /api/books	Seller	Manage seller-owned listings via REST
CategoryRestController	/api/categories	Public for GET, Admin for write	List, create, update, and delete categories
ExchangeRequestRestController	POST /api/requests, GET /api/requests/my	Authenticated	Create and view buyer requests
ExchangeRequestRestController	/api/seller/requests/**	Seller	Review and decide on incoming requests

Global REST errors are handled by GlobalRestExceptionHandler, which maps not-found, conflict, validation, bad-request, forbidden, and generic server errors to consistent JSON responses.

8. Testing and Quality Assurance

The project uses a mixed testing strategy to cover business logic, repository behavior, and REST controller behavior. According to the project inventory, the codebase currently includes 36 discovered test methods and uses JUnit 5, Mockito, Spring Boot Test, MockMvc, DataJpaTest, and an H2 test database configured in PostgreSQL mode.

8.1 Test Layers

Test Layer	Examples	Purpose
Service Unit Tests	AuthServiceImplTest, BookServiceImplTest, CategoryServiceImplTest, ExchangeRequestServiceImplTest, SellerApplicationServiceImplTest	Validate business rules in isolation using mocks
Repository Tests	BookRepositoryTest, CategoryRepositoryTest	Validate JPA queries and persistence behavior with H2
Integration Tests	BookRestControllerIntegrationTest, CategoryRestControllerIntegrationTest, ExchangeRequestRestControllerIntegrationTest	Validate HTTP endpoints, security, status codes, and persistence interactions

8.2 CI Behavior

- GitHub Actions runs service/unit tests first.
- Repository and integration tests run in a second stage.
- Render deployment is triggered only after both stages pass on pushes to main.

9. Dockerization and Deployment

9.1 Dockerization

- A multi-stage Dockerfile builds the Spring Boot application as a runnable JAR and packages it into a smaller runtime image.
- docker-compose.yml runs two services: app and postgres.
- The local app container maps port 9090 and connects to PostgreSQL using Docker service discovery.
- Environment variables are used for database connection details and seeded admin configuration.

9.2 Cloud Deployment

- The application is deployed to Render as a Docker-based web service.
- Render hosts the production web service and managed PostgreSQL database.
- GitHub Actions triggers the Render deploy hook after staged tests pass on main.
- The live deployed URL is https://sepm-book-exchange-platform.onrender.com

Purpose	Command / URL
Run local database only	docker compose up -d postgres
Run full stack in Docker	docker compose up --build
Public deployment	https://sepm-book-exchange-platform.onrender.com

10. Conclusion

The Book Exchange Platform satisfies the major academic goals of the Software Engineering and Project Management Lab by combining full-stack web development, role-based security, database design, automated testing, Dockerization, CI/CD, and cloud deployment into one coherent system.

The final solution demonstrates a practical workflow in which a buyer can register, apply for seller status, become an approved seller, publish book listings, receive requests, and manage those requests—while administrators retain moderation and category control.

This project can be extended further with richer analytics, image storage integration, notification systems, or additional reporting, but the current delivery already provides a complete, testable, and deployable semester project.

Appendix A. Important URLs

- Live deployment: https://sepm-book-exchange-platform.onrender.com
- Local home page: http://localhost:9090/
- Public catalog: http://localhost:9090/books
- Admin dashboard: http://localhost:9090/admin